

ARQUITECTURA DEL COMPUTADOR

INFORME PRÁCTICA DE LABORATORIO 1

Nombre: Victor Pinto, Ci: 31709216

Pregunta 1

¿Cómo se implementa la recursividad en MIPS32? ¿Qué papel cumple la pila (*sp*)?

La recursividad emplea procedimientos no hoja que constantemente se llaman a sí mismos. Poseen un caso base y un caso recursivo: el caso base se encarga de retornar el valor después de que se llegue a este a través de la disminución de un parámetro; el caso recursivo pregunta una condición que, de ser cumplida, se irá al caso base, o en su defecto se llamará nuevamente la función. Se usan las instrucciones `jal` para saltar al procedimiento y la instrucción `jr $ra` para retornar a la última dirección.

Al ser un procedimiento, se cargan en la pila (*\$sp*) los parámetros de entrada y la dirección de retorno. Debido a que la recursividad crea constantemente nuevas instancias, en cada una de ellas se accede a la pila para cargar los parámetros, que pertenecen a una instancia determinada.

Pregunta 2

¿Qué riesgos de desbordamiento existen? ¿Cómo mitigarlos?

Debido a la naturaleza de la recursión, el principal riesgo que existe es el desbordamiento de memoria o de la pila. Si se crean muchísimas instancias, la dirección de la pila apuntará a direcciones que no le fueron asignadas, causando un *stack overflow*. Para mitigar este problema, puede usarse recursión de cola, hacer uso de memoria dinámica, o simplemente optimizar el comportamiento de la recursión.

Pregunta 3

¿Qué diferencias encuentras entre una implementación iterativa y una recursiva en cuanto al uso de memoria y registros?

La diferencia más evidente está en que la implementación iterativa, a diferencia de la recursiva, no hace uso de la pila para ejecutarse. La iteración requiere registros para controlar los lazos, hacer uso de `j` para iterarse, y registros con los que operar aritméticamente y guardar el resultado. La recursión necesita hacer uso de la pila para guardar sus parámetros y los valores de retorno, a medida que se profundizan las instancias.

Pregunta 4

¿Qué diferencias encuentras entre los ejemplos académicos del libro y un ejercicio completo y operativo en MIPS32?

Los ejercicios académicos solo contienen el grueso de las instrucciones de MIPS. En simuladores como MARS, hay secciones importantes para un código real operativo, como la sección `.data` y `.text`, en donde se guardan en memoria los datos con los que se va a trabajar y la entrada a la función principal del programa respectivamente.

Pregunta 5

Elaborar un tutorial de la ejecución paso a paso en MARS.

Tutorial para implementación recursiva

1. Cargamos nuestro parámetro con la entrada deseada (n).
2. Hacemos `jal` (*jump and link*) para llamar la función recursiva.
3. Reservamos en pila tanto el parámetro con nuestra n como el registro de retorno `$ra`.
4. Se comprueba la condición: `bne` pregunta si n es igual al registro `$zero`.
5. Se saltará a la etiqueta `rec` si los registros no son iguales, siendo `rec` la parte del caso recursivo.
6. Se disminuye el parámetro ($n - 1$) y se llama a la función paridad con `jal`.
7. A medida que el parámetro disminuye, se crean nuevas instancias. Al llegar a $n = 0$, el registro `$v0` almacena 0. Se libera la pila, antes de devolverse a la última llamada usando `jr $ra`.
8. Cuando una de las instancias recibe la batuta, se carga de la pila el parámetro y el registro de retorno, se le suma 1 a lo retornado y a su vez se devuelve el `$v0` actualizado.
9. Se repite este proceso hasta llegar a la primera instancia original, que finalmente devuelve la salida.

Tutorial para implementación iterativa

1. Cargamos nuestro parámetro con la entrada deseada (n).
2. Hacemos `jal` (*jump and link*) para llamar la función iterativa.
3. Reservamos en pila tanto el parámetro con nuestra n como el registro de retorno `$ra`.
4. Declaramos una variable para iterar en 0.

5. Hacemos un lazo llamado **while** que pregunta constantemente si $n > 0$. De no serlo, saltará a la salida.
6. Nuestro iterador crece, y si este llega a ser mayor que 1, se resetea a 0.
7. El parámetro disminuye, y cuando llegue a 0, retornamos el valor de nuestro iterador.

Pregunta 6

Justificar la elección del enfoque (iterativo o recursivo) según eficiencia y claridad en MIPS.

Para valores muy grandes la elección se debe dirigir a la implementación iterativa, pues es la más óptima en cuanto al consumo de recursos de espacio. El desarrollo de la iteración es más comprensible al momento de leer código, a diferencia de la recursión que requiere un mayor nivel de abstracción, y en donde el uso de la pila puede ser difuso. La iteración usa operaciones aritméticas básicas e instrucciones de MIPS bien conocidas.

Pregunta 7

Análisis y Discusión de los Resultados

Casos básicos

Con casos simples, en donde la entrada son números pequeños, el comportamiento de ambas implementaciones es el esperado y casi iguales: la salida arroja 1 para números impares y 0 para números pares.

Casos especiales

Hay una diferencia sustancial en la ejecución de ambas implementaciones cuando la entrada es un número más grande. Se ingresó el número 1505001, el cual está dentro del rango de una palabra en MIPS.

- **Implementación iterativa:** La función tarda unos cuantos segundos en arrojar el resultado, pero se ejecuta con normalidad y finaliza el programa sin *bugs*.
- **Implementación recursiva:** La terminal de MARS manda el siguiente mensaje:
`paridad_recur.asm line 29: Runtime exception at 0x0040002c: address out of range 0x7fbffffc`

Explicación

La función iterativa resta de 1 en 1 el número, repitiéndose constantemente en el lazo, lo cual no produce ningún desbordamiento en la memoria, pero lleva más tiempo su finalización.

Ahora, la función recursiva no se compila correctamente y causa un error de desbordamiento. El mensaje `address out of range 0x7fbffffc` ejemplifica lo que se había discutido antes: el puntero de la pila, al quedarse sin espacio, accede a una dirección

inválida fuera de su rango, debido a la alta cantidad de instancias creadas que un número así de grande requiere.